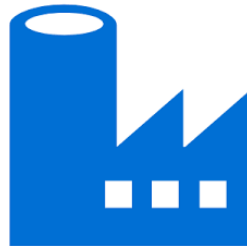




Web: inceptez.com Mail: info@inceptez.com Call: 7871299810, 7871299817

Azure Synapse Analytics -Project Walkthrough



HR Analytics – Project Overview

The objective of this project is to build a scalable and analytics-driven HR data platform using Azure Synapse Analytics. The platform will consolidate employee-related datasets from multiple HR systems and generate metrics

Metrics Outcome

Bronze (Raw Layer)

- Raw ingestion from:
 - Employee system (HRMS)
 - Performance system
 - Survey tool
 - Exit interviews
- Stored as CSV in ADLS

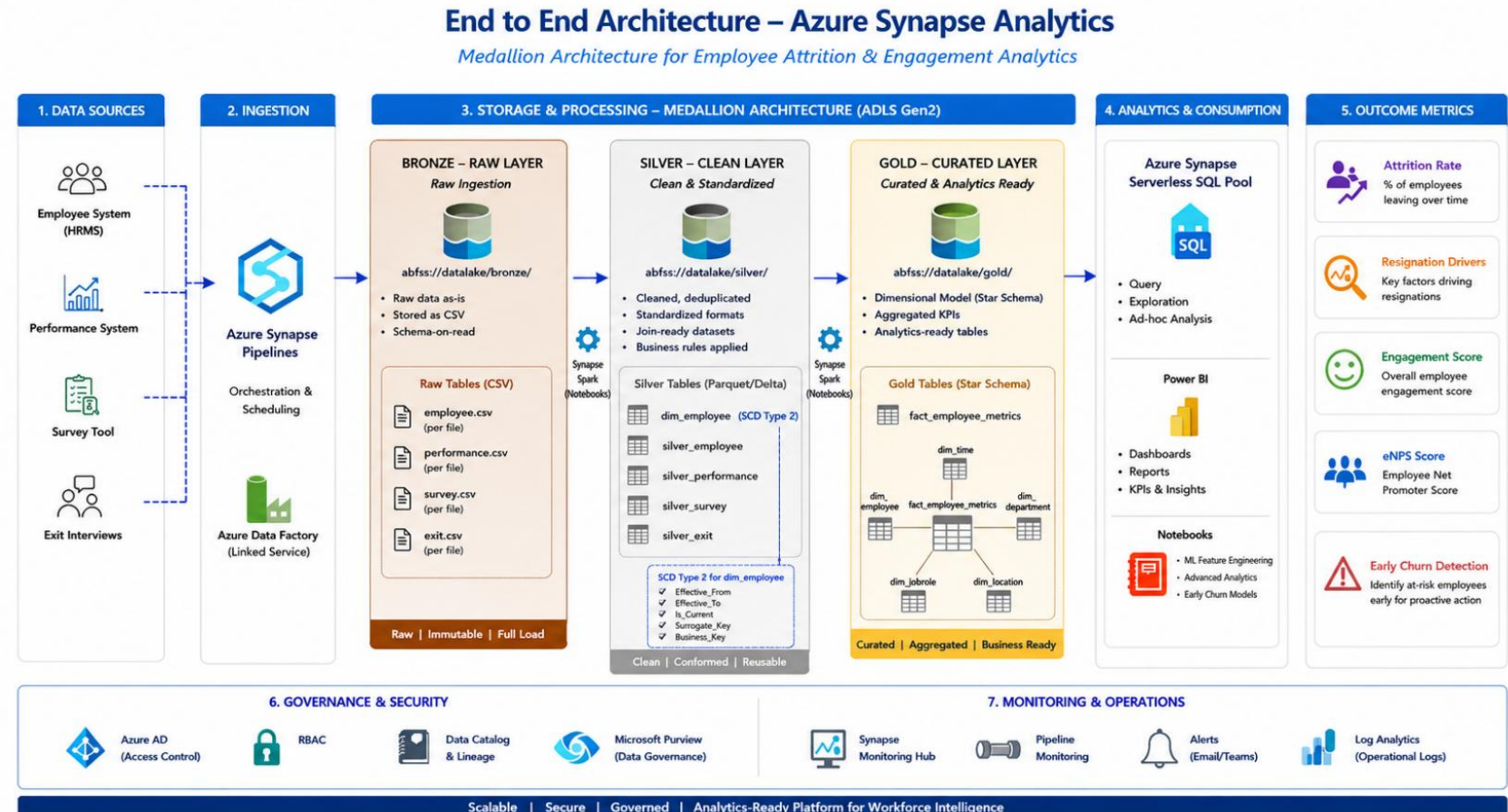
Silver (Clean Layer)

- Clean, deduplicated, standardized
- Join-ready datasets
- Apply business rules

Gold (Curated Layer)

- Dimensional model (Star Schema)
- Aggregated KPIs
- Analytics-ready tables

- Predict **employee attrition**
- Identify **resignation drivers**
- Measure **employee engagement & eNPS**
- Detect **early churn signals**



Azure Synapse Analytics - Overview

What is Azure Synapse Analytics?

Azure Synapse Analytics is Microsoft's enterprise analytics platform for working with **data integration, data warehousing, big data, and analytics in one workspace**.

Microsoft describes it as a service that brings together enterprise data warehousing and big data analytics, with options to query data using either serverless or dedicated resources.

Azure Synapse Analytics is a **unified analytics platform** from Microsoft that brings together:

- Data warehousing
- Big data analytics
- Data integration
- Data visualization

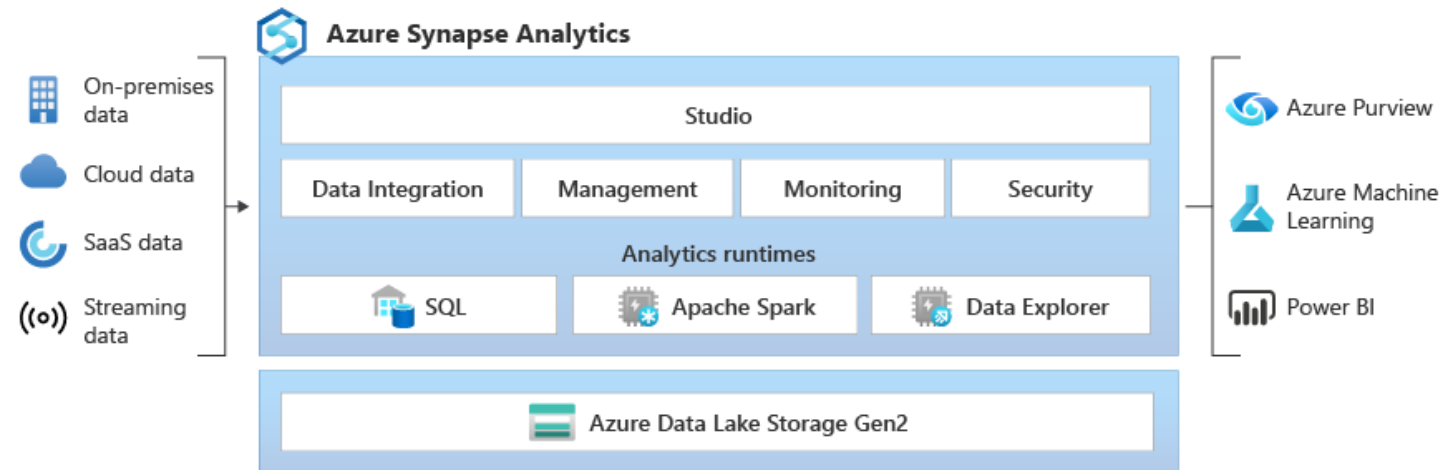
all in **one single service**

ADF vs Databricks vs Synapse

Databricks = Big data + streaming + Spark + orchestration

Azure Data Factory = mainly data movement and orchestration

Azure Synapse Analytics = orchestration + SQL analytics + Spark + warehouse + unified analytics workspace.



Components in Azure Synapse

Azure Synapse Analytics is made up of several integrated components that work together for **data ingestion, processing, storage, and analytics**.

Synapse Studio (UI Layer)

Web-based workspace to manage everything

- Build pipelines
- Write SQL queries
- Create Spark notebooks
- Monitor jobs

Synapse Pipelines (ETL/ELT)

Similar to Azure Data Factory

- Data ingestion from multiple sources
- Scheduling & orchestration
- Supports 100+ connectors

Apache Spark Pools (Big Data Processing)

Built-in Spark engine

- PySpark, Scala, SQL support
- Data transformation
- Machine Learning

SQL Pools (Data Warehouse Layer)

Dedicated SQL Pool

- Provisioned (fixed resources)
- High-performance data warehouse

Serverless SQL Pool

- On-demand querying
- Query data directly from Data Lake

Data Lake Integration

Works with

Azure Data Lake Storage Gen2

- Stores raw, curated, and processed data
- Acts as central storage

Integration with External Services

- Microsoft Power BI → Reporting
- Azure Machine Learning → ML models
- Azure Cosmos DB → Real-time data

Environment Setup & Workspace

Phase 1: The Prerequisites

1. An Azure Subscription
2. Create Resource Group:

Provide:

- Name: rg-data-project
- Region: (e.g., Central India)

3. Create Storage Account (Data Lake)

Create containers:

- raw
- processed
- curated

Phase 2: Creating the Synapse Workspace

1. In top search bar type “Azure Synapse”
2. Click “**Azure Synapse Analytics**” under service
3. Fill the details
 1. Subscription
 2. Resource group
 3. Workspace name
 4. Region
4. Configure Storage
 1. This is where data, notebooks, logs, etc. will be stored
5. Security & Networking Tab: set an admin password for the Serverless SQL pool
6. Review + Create
7. Launch Synapse Studio

Serverless SQL Pool

A Serverless SQL Pool in Azure Synapse Analytics allows you to query data directly from your Data Lake without creating or managing any infrastructure.

Core Characteristics

- **No Infrastructure Management**

Automatically scales compute; no need to provision or manage clusters

- **Query Data Directly (In-Place)**

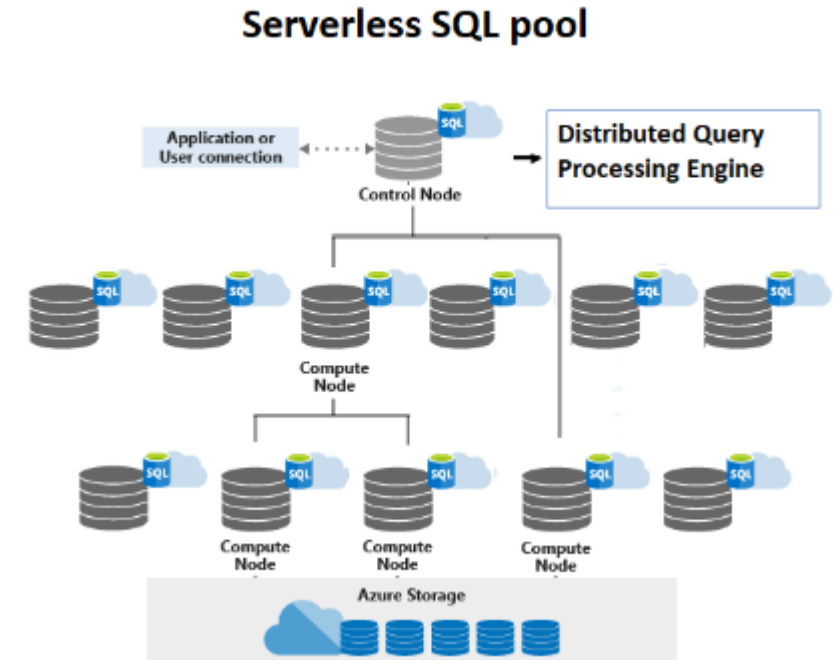
Uses `OPENROWSET` to read data from sources like **ADLS Gen2** or Cosmos DB without loading

- **Supports Multiple Formats**

Works with CSV, JSON, Parquet, and Delta Lake

- **Uses Standard T-SQL**

Can be accessed using SQL tools like Power BI, Tableau, or SSMS



Querying data with Serverless SQL Pool

OPENROWSET is used to **read data directly from external sources** (like files or other databases) **without creating external tables**.

- OPENROWSET() Function to query data
- We can use OPENROWSET() function with or without DATA_SOURCE parameter
- We will use OPENROWSET() after FROM Clause in T-SQL query
- The OPENROWSET function is not supported in dedicated SQL pool.

```
SELECT * FROM OPENROWSET()
```

Mandatory parameters for OPENROWSET()

```
SELECT TOP 100 * FROM OPENROWSET(  
  *BULK “,  
  *FORMAT = ‘CSV or PARQUET or DELTA’,  
  *PARSER_VERSION = ‘1.0 or 2.0’  
) AS [result]
```

```
SELECT *  
FROM OPENROWSET(  
  BULK  
  'https://storageaccount.blob.core.windows.net/container/file.csv',  
  FORMAT = 'CSV',  
  PARSER_VERSION = '2.0',  
) AS rows;
```

Serverless Pool - Components

In Azure SQL Database, these components form the structural and security framework for managing data locally and across external sources

External Table

External tables allow you to **query data stored outside Azure SQL Database** as if it were a local table.

- Azure Blob Storage
- ADLS
- Azure Cosmos db

External Resources

External resources are the **dependencies required to define external tables**.

- External Data Source
- External File Format
- Database Scoped Credential

Views

A virtual table based on a SQL query.

- Simplify complex queries
- Abstract logic
- Restrict data access
- Reuse logic

Schemas

A **logical container** for database objects (tables, views, procedures).

- Organize objects
- Separate environments (dev, staging, prod)
- Control access at schema level

Security

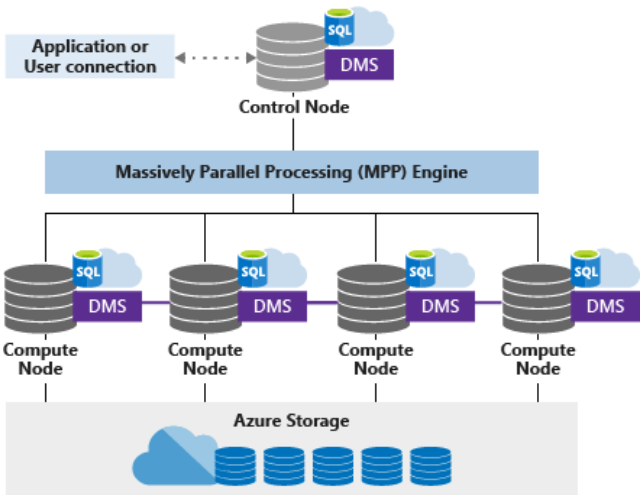
Azure SQL Database employs a layered security model to protect data.

- Authentication
- Authorization
- Granular Control

Dedicated SQL Pool

Dedicated SQL Pool is a **provisioned, MPP (Massively Parallel Processing)** data warehouse where data is **physically stored inside Synapse** and compute resources are **reserved**.

- Formerly called **SQL DW (SQL Data Warehouse)**
- Data is stored in **distributed tables**
- Compute is **pre-allocated (DWU)**
- Designed for **large-scale analytics**
- Uses **MPP architecture**
- Supports **materialized views, table distribution, Index**



- Use when:**
- Large data warehouse
 - Complex joins
 - Aggregations
 - Power BI dashboards
 - ETL curated layer
 - High concurrency reporting

Feature	Dedicated SQL Pool	Serverless SQL Pool
Data storage	Inside Synapse	In Data Lake
Compute	Provisioned (DWU)	On-demand
Performance	Very fast (MPP)	Slower
Cost	Fixed (running compute)	Pay per query
Materialized views	✔ Yes	✖ No
Insert/Update/Delete	✔ Yes	Limited
Best for	Data warehouse	Data lake querying

Index

Index Type	Description	Best Use Case	Performance	Storage	Default
Clustered Columnstore Index (CCI)	Data stored in compressed column format	Large fact tables	Best for analytics	Lowest	Yes
Heap	No index, raw table	Staging / fast load	Poor for queries	Normal	No
Clustered Index (Rowstore)	Traditional B-tree index	Small tables / lookup	Good for point queries	Higher	No

Distribution

Distribution Type	How Data is Distributed	Best Use Case	Table Size	Join Performance	Load Speed	Data Movement	Example
HASH	Based on hash of a column	Large fact tables	Large (>60M rows)	Best (if join on same key)	Medium	Minimal	DISTRIBUTION = HASH(customer_id)
ROUND_ROBIN	Evenly & randomly distributed	Staging / temp tables	Any	Poor–Medium	Fastest	High (shuffle)	DISTRIBUTION = ROUND_ROBIN
REPLICATE	Full table copied to all nodes	Small dimension tables	Small (<2GB)	Best	Slow	None	DISTRIBUTION = REPLICATE

Data Load Options

Method	Description	Best Use Case	Performance	Notes
COPY INTO	Load directly from ADLS into table	Recommended & modern	Fast	Simple & parallel
INSERT INTO...SELECT (External table)	Load via external table	Incremental loads	Medium	Flexible
CREATE TABLE AS SELECT (CTAS)	Create table + load data	Initial loads	Very Fast	Best for transformations
ADF / Synapse Pipeline Copy	GUI-based copy	ETL pipelines	Fast	No-code option
BULK INSERT (limited)	Direct file load	Small loads	Slow	Rarely used

External Table

Use Dedicated External Table

- Loading data into warehouse
- Building curated layer
- High performance required
- Repeated reporting queries

Use Serverless External Table

- Query raw data lake
- Ad-hoc analysis
- No data loading needed
- Cost optimization

Feature	Dedicated SQL Pool External Table	Serverless SQL Pool External Table
Compute	Provisioned (DWU)	On-demand
Query engine	MPP distributed engine	Serverless query engine
Performance	Faster for repeated queries	Slower but flexible
Cost	Fixed compute cost	Pay per TB scanned
Best use case	Data warehouse staging	Ad-hoc data lake querying
Insert into internal table	✔ Yes	✗ No
Materialized views	✔ Supported	✗ Not supported
Data format	CSV, Parquet, ORC	CSV, Parquet, ORC, Delta Lake

Spark Pool in Azure Synapse

A **Spark Pool** in **Azure Synapse Analytics** is a **serverless Apache Spark cluster** used to **process and transform large volumes of data** using languages like **PySpark, Spark SQL, Scala, and .NET**.

Key Features:

- **Managed Serverless Experience**

Azure manages infrastructure automatically. No need to configure VMs or install software.

- **Autoscaling**

Cluster automatically **scales up or down** based on workload demand.

- **Cost Efficiency**

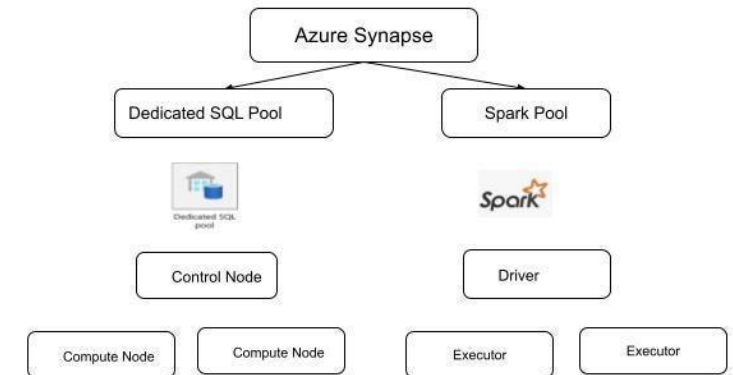
Idle Spark sessions **auto-shut down** using Time-to-Live (TTL) to reduce cost.

- **Multi-user Support**

Multiple users can **run notebooks simultaneously** on the same Spark pool.

- **Language Support**

Supports **PySpark, Scala, Spark SQL, and .NET (C#)**.



Spark Pool Components

1. Synapse Workspace

- Entry point for development
- Create notebooks, pipelines, jobs
- Connects to Spark pool

2. Spark Pool (Cluster)

When we run a notebook, Synapse creates a **Spark cluster**:

Driver Node

- Controls Spark job
- Creates execution plan
- Coordinates executors

Executor Nodes

- Process data in parallel
- Run tasks
- Read/write from Data Lake

3. Azure Data Lake Storage

- Source & destination data
- Spark reads and writes here
- Usually ADLS Gen2

4. Notebooks / Jobs

- PySpark notebook
- Spark SQL
- Scala notebook

This triggers cluster creation.

Notebook Magic Commands & Markdown Format

Command	Purpose
%%pyspark	Run PySpark (Python) code
%%spark	Run Scala code
%%sql	Execute Spark SQL queries
%%csharp	Run .NET for Apache Spark
%%md	Write Markdown text

Headings	Bold and Italic	Unordered List	Ordered List	Link	Image
# H1	Bold: **text**	- Item 1	1. First item	[Click here](url)	![PySpark Logo](URLpath)
## H2	Italic: <i>*text*</i>	- Item 2	2. Second item		
### H3	Bold & Italic: <i>***text***</i>	- Sub-item 1	1. Sub-item 1		
#### H4		- Sub-item 2	2. Sub-item 2		

mssparkutils in Azure Synapse (Databricks dbutils Equivalent)

mssparkutils is a built-in utility library in Synapse used for **file operations, secrets, notebook chaining, and env info**

1. File System Utilities (mssparkutils.fs)

Used to interact with ADLS Gen2 files and folders.

Command	Description	Example
ls	List directory	mssparkutils.fs.ls("abfss://...")
cp / mv	Copy / Move files	mssparkutils.fs.cp(src, dest)
mkdirs	Create folder	mssparkutils.fs.mkdirs("/new_folder")
rm	Delete file/folder	mssparkutils.fs.rm("/old", True)
head	Preview file	mssparkutils.fs.head("file.txt",100)

2. Mount Storage

Mount ADLS container as a local path

```
mssparkutils.fs.mount("abfss://data@mystorage.dfs.core.windows.net/",
"/mnt/processed",{ "linkedService":"MySynapseLinkedService"})
print(mssparkutils.fs.ls("/mnt/processed"))
```

3. Credentials & Secrets (mssparkutils.credentials)

Safely retrieve secrets from Key Vault or Linked Service.

Get secret from Key Vault

```
token = mssparkutils.credentials.getSecret("MyKeyVaultName","StorageKey")
conn = mssparkutils.credentials.getConnectionStringOrEmpty("MySQLLinkedService")
```

4. Notebook Chaining (mssparkutils.notebook)

Run one notebook from another (ETL orchestration).

```
mssparkutils.notebook.run("ChildNotebook",900,{"run_date": "2026-04-10"})
```

5. Runtime Context (mssparkutils.env)

Get environment details for logging/debugging.

```
user_email = mssparkutils.env.getUserEmail()
workspace_name = mssparkutils.env.getWorkspaceName()

print(f"Job running in {workspace_name} by {user_email}")
```

Databricks	Synapse
dbutils.fs	mssparkutils.fs
dbutils.secrets	mssparkutils.credentials
dbutils.notebook.run	mssparkutils.notebook.run
dbutils.widgets	parameters in notebook.run
dbutils.fs.mount	mssparkutils.fs.mount



Web: inceptez.com Mail: info@inceptez.com Call: 7871299810, 7871299817

Azure Functions



Azure Functions - Overview

Azure Functions (often called *Azure Cloud Functions*) is a **serverless service** in Azure where write small pieces of code that run automatically based on events.

Just write the function (code), and Azure handles:

- Infrastructure
- Scaling
- Availability

It runs only when triggered by an event.

Core Concepts of Azure Functions:

- **Code (Function Logic)**

The business logic written in languages like Python, C#, or JavaScript.
It defines what action the function performs when triggered.

- **Events (Triggers)**

A trigger initiates the function execution based on an event.
Each function has exactly one trigger, such as HTTP, timer, or queue events.

- **Data (Bindings)**

Bindings connect the function to external services with minimal code.
They handle input and output operations with services like Azure Cosmos DB and Azure Blob Storage.

Azure Functions – Trigger Types

Trigger Type	Description	Example Use Case
HTTP Trigger	Runs when an HTTP request is received	Create a REST API to add or fetch employee data
Blob Trigger	Fires when a file is added/updated in Azure Blob Storage	Resize or validate images on upload
Timer Trigger	Executes on a schedule using CRON	Generate daily reports at 2 AM
Queue Trigger	Runs when a message is added to a queue	Process background jobs like order handling
Event Hub Trigger	Processes streaming data from Azure Event Hubs	Analyze real-time IoT or telemetry data
Cosmos DB Trigger	Executes on data changes in Azure Cosmos DB	Update dashboards when new data is inserted
Event Grid Trigger	Responds to events from Azure Event Grid	Send alerts when resources are created

Azure Functions – Common Use Case

Use Case	Description	Example
Web API Backends	Build lightweight, scalable APIs without managing servers	Create an API to manage employee data or orders
File Processing Automation	Automatically process files when uploaded to storage	Resize images or validate CSV files in Azure Blob Storage
Scheduled Jobs	Run tasks at fixed intervals using timer triggers	Daily report generation or database cleanup at night
Event-Driven Integration	Connect and automate workflows between systems	Process messages from Azure Service Bus and store in DB
Real-Time Data Processing	Handle streaming or high-volume event data	Analyze IoT data using Azure Event Hubs
Data Transformation (ETL/ELT)	Transform and move data between systems	Integrate with Azure Data Factory for pipelines
API to Storage Ingestion	Fetch data from external APIs and store it in cloud storage	Call a REST API and save the response as JSON/CSV in Azure Blob Storage
Notification Systems	Send alerts or notifications based on events	Email/SMS alerts when a new order is placed
Background Jobs	Run asynchronous or long-running backend tasks	Process bulk data uploads or generate invoices
Serverless Workflows	Orchestrate multiple steps in a workflow	Multi-step approval process using chained functions
Monitoring & Automation	React to system events and automate actions	Trigger alerts using Azure Event Grid

Azure Functions – Authentication Types

Feature / Aspect	Anonymous	Function	Admin
Key Required	No	Yes (Function Key)	Yes (Host/Admin Key)
Access Scope	Public (anyone)	Specific function only	Entire Function App
Security Level	Low	Medium	High (Very sensitive)
How to Access	Direct URL	?code=<function_key> or header	?code=<admin_key>
Key Type	Not required	Function-specific key	Master/host key
Exposure Risk	Very high	Moderate	Extremely high (full control)
Typical Use Cases	Public APIs, webhooks, demos	Internal APIs, microservices	Admin APIs, management operations
Recommended Usage	Rare (only if safe)	Most common choice	Avoid unless necessary